Estimating π with Monte Carlo Methods

Elementary Applications of Variance Reduction Techniques

Brenno Vaccari

Abstract

This report presents a small numerical experiment on the estimation of π through Monte Carlo integration. Its purpose is educational: at the undergraduate level, it illustrates some elementary applications of techniques that appear in much more general and sophisticated settings in the Monte Carlo literature. No claim of originality or research contribution is made. The standard Monte Carlo method, antithetic sampling, the control variates method, and importance sampling are compared. For each method, empirical bias, empirical variance, RMSE, approximate confidence-interval coverage, and execution time are examined. The essential theoretical results are mainly recalled from Emmanuel Gobet's textbook [1], in particular from the chapters devoted to convergence and variance reduction.

Contents

1 Project Aim **2**

2 Minimal Theoretical Background **2**
2.1 Monte Carlo as Estimation of an Expected Value 2
2.2 A Non-Asymptotic Bound: Chebyshev's Inequality 3
2.3 Antithetic Sampling 3
2.4 Control Variates 4
2.5 Importance Sampling 4

3 Numerical Implementation **5**
3.1 Libraries Used 5
3.2 Experimental Protocol 5

4 Results **6**
4.1 Standard Monte Carlo 6
4.2 Antithetic Sampling 7
4.3 Control Variates 7
4.4 Importance Sampling 8

5 Comparison of the Methods **8**
5.1 RMSE Comparison 8
5.2 Theoretical and Empirical Reduction Factors 9
5.3 Execution-Time Comparison 9
5.4 Final Results Expressed as Estimates of π 9
5.5 Confidence-Interval Coverage 10

6 Discussion and Limitations **10**

7 Conclusions **11**

1 Project Aim

The project is intentionally limited in scope. The idea is to start from an elementary integral whose exact value is known and use it as a test case for several probabilistic methods of numerical approximation. This makes it possible to separate two aspects: the basic theory required to define the estimators, and the numerical behavior observed in repeated experiments.

The integral considered is

$$I = \int_0^1 \frac{1}{2\sqrt{1-(x/2)^2}}\,dx. \tag{1}$$

With the substitution $t = x/2$, we obtain

$$I = \int_0^{1/2} \frac{1}{\sqrt{1-t^2}}\,dt = \arcsin\left(\frac{1}{2}\right) = \frac{\pi}{6}.$$

Therefore, any estimator $\widehat{I}$ immediately provides an estimator of π:

$$\widehat{\pi} = 6\widehat{I}. \tag{2}$$

Consequently, bias, RMSE, and the margin of error are multiplied by 6, while the variance is multiplied by 36.

2 Minimal Theoretical Background

2.1 Monte Carlo as Estimation of an Expected Value

Let $U \sim \mathcal{U}([0,1])$ and define

$$f(x) = \frac{1}{2\sqrt{1-(x/2)^2}}.$$

Then

$$I = \mathbb{E}[f(U)].$$

If $U_1, \ldots, U_M$ are independent and uniformly distributed on $[0,1]$, the standard Monte Carlo estimator is

$$\widehat{I}_M = \frac{1}{M}\sum_{i=1}^{M} f(U_i). \tag{3}$$

It is unbiased (i.e. $\mathbb{E}[\widehat{I}_M] = I$) and

$$\mathrm{Var}(\widehat{I}_M) = \frac{\mathrm{Var}(f(U))}{M}. \tag{4}$$

Hence its root mean squared error decreases at rate $M^{-1/2}$: in particular,

$$\mathrm{RMSE}(\widehat{I}_M) = \sqrt{\mathrm{Var}(\widehat{I}_M)} = \frac{\sigma}{\sqrt{M}} = O(M^{-1/2}). \tag{5}$$

The law of large numbers and the central limit theorem provide the theoretical justification for convergence and asymptotic confidence intervals; see [1, Ch. 2].

In this example, it is also possible to compute explicitly

$$\mathbb{E}[f(U)^2] = \int_0^1 \frac{1}{4(1-x^2/4)}\,dx = \frac{1}{4}\log 3,$$

and therefore

$$\sigma^2 := \mathrm{Var}(f(U)) = \frac{1}{4}\log 3 - \frac{\pi^2}{36} \approx 4.97394 \times 10^{-4}. \tag{6}$$

In the notebook, the margin of error is computed as

$$1.96\,\frac{\sigma}{\sqrt{M}}, \tag{7}$$

which corresponds to a nominal two-sided 95% confidence interval based on the normal approximation.

2.2 A Non-Asymptotic Bound: Chebyshev's Inequality

By Chebyshev's Inequality, for every $\varepsilon > 0$,

$$\mathbb{P}\left(|\widehat{I}_M - I| \geq \varepsilon\right) \leq \frac{\sigma^2}{M\varepsilon^2}. \tag{8}$$

Therefore, to require this probability to be at most p, it is sufficient to choose

$$M \geq \frac{\sigma^2}{p\varepsilon^2}. \tag{9}$$

This bound is robust but generally conservative. The notebook visualizes the dependence of the required number of iterations on ε and p.

2.3 Antithetic Sampling

Antithetic sampling replaces each sample U with the pair U and $1-U$. Consider

$$Y_{\mathrm{AV}} = \frac{f(U) + f(1-U)}{2}. \tag{10}$$

Since U and $1-U$ have the same uniform distribution, $\mathbb{E}[Y_{\mathrm{AV}}] = I$. Moreover,

$$\mathrm{Var}(Y_{\mathrm{AV}}) = \frac{1}{2}\sigma^2 + \frac{1}{2}\,\mathrm{Cov}(f(U), f(1-U)). \tag{11}$$

If the covariance is negative, the variance decreases. In this case, f is increasing, while $u \mapsto f(1-u)$ is decreasing, so the antithetic mechanism is favorable. The technique is discussed in [1, Sec. 3.1].

To compare the method correctly with standard Monte Carlo, the total number N of evaluations of f is fixed. One antithetic pair requires two evaluations, so $N/2$ pairs are generated. The theoretical variance-reduction factor at equal numbers of function evaluations obtained in the notebook is

$$\frac{\mathrm{Var}(\widehat{I}_N)}{\mathrm{Var}(\widehat{I}^{\mathrm{AV}}_{N/2})} \approx 5.766. \tag{12}$$

Therefore, by naming

$$\mathrm{g}(U) = \frac{\mathrm{f}(U) + \mathrm{f}(1-U)}{2}, \tag{13}$$

and applying the Monte Carlo method to $\mathrm{g}(U)$, the root mean squared error is reduced by a factor of approximately $\sqrt{5.766} \approx 2.40$, while retaining the same $O(N^{-1/2})$ convergence rate (see (5)). The same reasoning applies to the other methods, leading to analogous estimates of the improvement.

2.4 Control Variates

A random variable Z with known mean and correlated with $f(U)$ is introduced to lower the variance of the method. In the notebook, the choice is

$$Z = U - \frac{1}{2}, \qquad \mathbb{E}[Z] = 0. \tag{14}$$

The estimator is

$$\widehat{I}_M^{\mathrm{CV}} = \frac{1}{M} \sum_{i=1}^{M} \left[f(U_i) - \beta Z_i \right]. \tag{15}$$

The value that minimizes the variance is

$$\beta^\star = \frac{\mathrm{Cov}(f(U), Z)}{\mathrm{Var}(Z)} = \frac{\mathrm{Cov}(f(U), Z)}{\mathbb{E}[Z^2]}. \tag{16}$$

In the notebook's numerical calculation,

$$\beta^\star \approx 0.0737976556. \tag{17}$$

With this choice, the theoretical variance-reduction factor is

$$\frac{\mathrm{Var}(f(U))}{\mathrm{Var}(f(U) - \beta^\star Z)} \approx 11.420. \tag{18}$$

The principle and the optimal choice of the coefficient are described in [1, Sec. 3.3].

2.5 Importance Sampling

Importance sampling changes the distribution from which samples are drawn. If X has a density q that is positive on $[0, 1]$, then

$$I = \int_0^1 f(x)\, dx = \mathbb{E}_q \left[\frac{f(X)}{q(X)} \right]. \tag{19}$$

The estimator is therefore

$$\widehat{I}_M^{\mathrm{IS}} = \frac{1}{M} \sum_{i=1}^{M} \frac{f(X_i)}{q(X_i)}, \qquad X_i \sim q. \tag{20}$$

Its per-sample variance is

$$\int_0^1 \frac{f(x)^2}{q(x)}\, dx - I^2. \tag{21}$$

The general theory is reviewed in [1, Sec. 3.4].

In the notebook, the choice is restricted to the linear family

$$q_a(x) = 1 + a \left(x - \frac{1}{2} \right), \tag{22}$$

which is already normalized on $[0, 1]$. The parameter a is chosen numerically by minimizing the variance within this family. The stationarity condition is solved by bisection and gives

$$a \approx 0.1394130588. \tag{23}$$

Hence

$$q_a(x) = 0.1394130588\, x + 0.9302934706.$$

The cumulative distribution function is inverted explicitly, allowing $X \sim q_a$ to be generated from a uniform random variable. The theoretical variance-reduction factor obtained is

$$\frac{\mathrm{Var}(f(U))}{\mathrm{Var}_q(f(X)/q(X))} \approx 11.487. \tag{24}$$

3 Numerical Implementation

3.1 Libraries Used

The notebook intentionally uses a simple set of Python libraries.

- `NumPy` is used to create and manipulate numerical arrays, for example to store RMSE values and iteration counts. [2]
- `Matplotlib` is used for the plots, especially the log–log plots and the three-dimensional surface related to Chebyshev's inequality. [4]
- The standard `random` module generates pseudo-random samples uniformly distributed on $[0, 1]$ through `random.uniform`. It is also used to fix a seed to enable experimental reproducibility
- The `math` module provides scalar functions such as square roots, π, and elementary operations.
- The `time` module measures elapsed time during the simulation series. These times depend on the machine and execution environment and should not be interpreted as universal benchmarks.
- `scipy.integrate.quad` performs adaptive numerical integration and is used to compute theoretical benchmark quantities such as means, variances, and covariances, as well as to determine the parameters $\beta^\star$ and a. In the notebook, the default tolerances are approximately 1.49×10^{-8}. [3]

The code structure prioritizes pedagogical readability: the estimators are implemented with explicit Python loops rather than full NumPy vectorization. This makes the correspondence between the mathematical formula and the algorithm immediate, but it reduces computational efficiency.

3.2 Experimental Protocol

For each method and for each sample size

$$N \in \{10^2, 10^3, 10^4, 10^5, 10^6\},$$

100 independent replications are performed. For the antithetic method, N denotes the total number of evaluations of f, so the number of pairs is $N/2$.

From the 100 replications, the following quantities are computed:

- empirical bias;
- empirical variance;
- sample variance;
- root mean squared error (RMSE);
- theoretical margin of error based on 1.96 standard deviations;
- empirical percentage of replications falling within the nominal interval.

The following tables report exactly the results already stored in the analyzed file.

4 Results

4.1 Standard Monte Carlo

The log–log plot shows the decrease of the RMSE as the number of iterations increases and compares it with the theoretical slope $N^{-1/2}$. In all RMSE plots, the curve $N^{-1/2}$ is included solely as a reference for the convergence rate, not as a prediction of the absolute RMSE values.

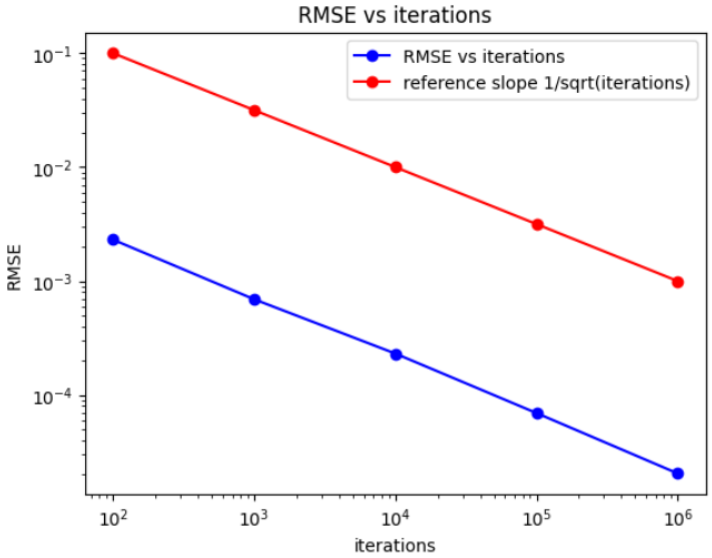


Figure 1: RMSE of the standard Monte Carlo method and the $N^{-1/2}$ reference. Plot extracted from the notebook.

The second plot visualizes, using a logarithmic scale for the required number of iterations, the bound obtained from Chebyshev's inequality.

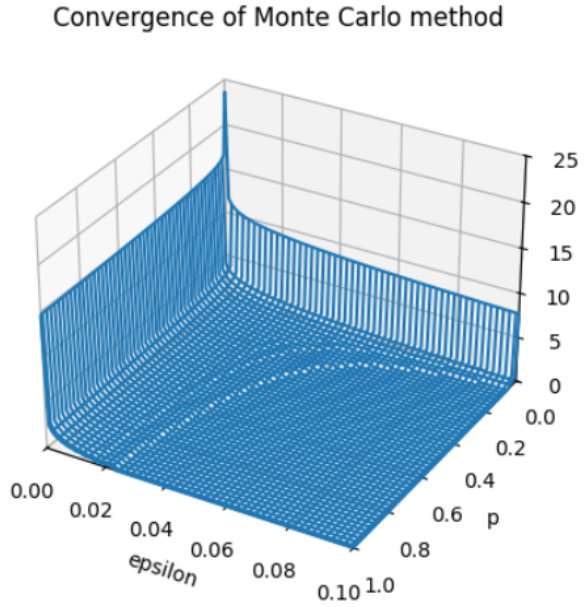


Figure 2: Number of iterations sufficient according to Chebyshev's inequality as the tolerance ε and probability threshold p vary. Plot extracted from the notebook.

4.2 Antithetic Sampling

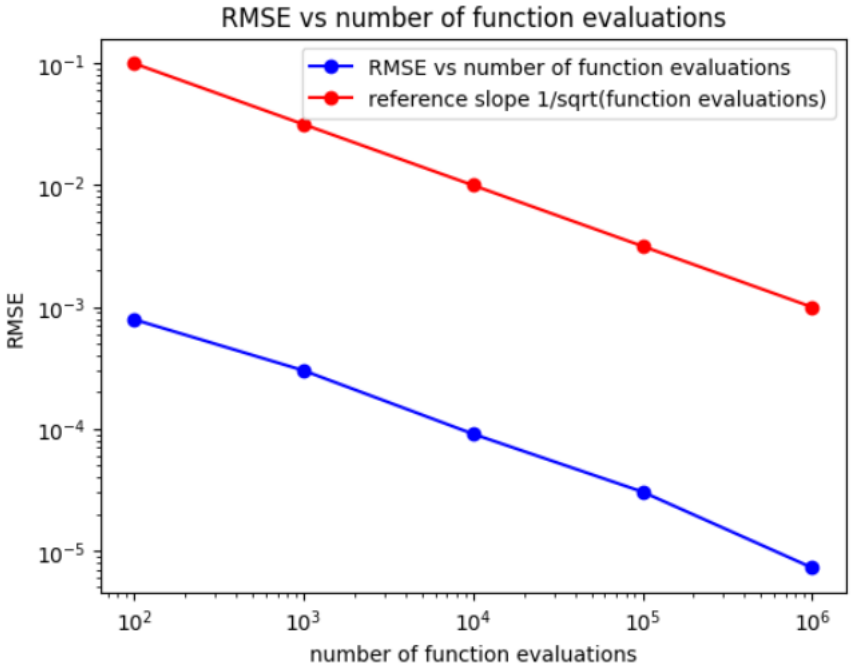


Figure 3: RMSE of antithetic sampling as a function of the total number of evaluations of f.

The advantage is already visible at $N = 100$: the RMSE decreases from approximately 2.306×10^{-3} for the standard method to 7.900×10^{-4}. At $N = 10^6$, the RMSE is approximately 7.246×10^{-6}, compared with 2.048×10^{-5} for the standard method.

4.3 Control Variates

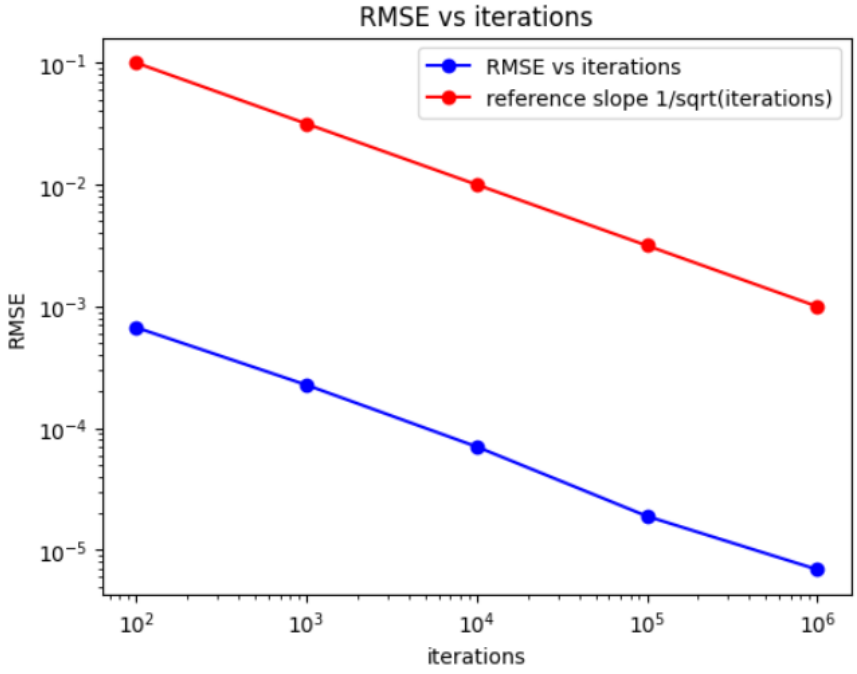


Figure 4: RMSE of the control variate method.

The method produces a more pronounced improvement than antithetic variates. For $N = 10^6$, the observed RMSE is approximately 6.924×10^{-6}.

4.4 Importance Sampling

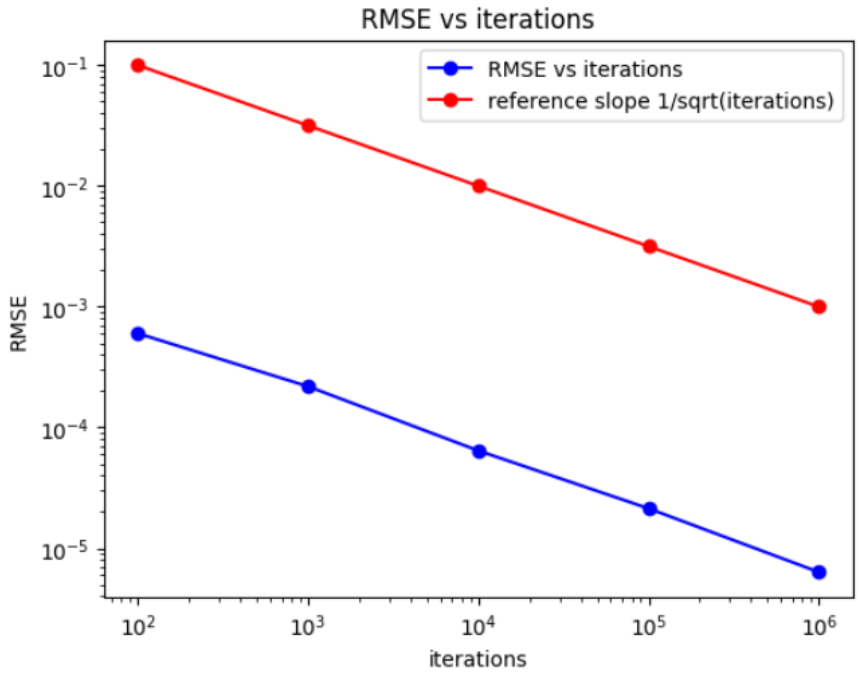


Figure 5: RMSE of importance sampling with a linear density optimized within the chosen family.

From a purely statistical perspective, this method gives the lowest RMSE in the stored simulation: approximately 6.345×10^{-6} for $N = 10^6$. However, the advantage over the control variate method is small, while the observed computational cost is much higher.

5 Comparison of the Methods

5.1 RMSE Comparison

Table 1 directly compares the RMSE values for the integral $I = \pi/6$ at equal numbers of evaluations of f.

Table 1: Empirical RMSE of $\widehat{I}$ over 100 replications.

N	Standard	Antithetic	Control variate	Importance sampling
100	2.306×10^{-3}	7.900×10^{-4}	6.712×10^{-4}	6.011×10^{-4}
1 000	6.913×10^{-4}	3.009×10^{-4}	2.265×10^{-4}	2.186×10^{-4}
10 000	2.308×10^{-4}	9.078×10^{-5}	7.051×10^{-5}	6.424×10^{-5}
100 000	6.936×10^{-5}	3.038×10^{-5}	1.896×10^{-5}	2.135×10^{-5}
1 000 000	2.048×10^{-5}	7.246×10^{-6}	6.924×10^{-6}	6.345×10^{-6}

All methods show a decay compatible with the $N^{-1/2}$ rate: variance-reduction techniques do not change the asymptotic order of classical Monte Carlo, but they reduce the multiplicative constant. This is precisely the advantage visible in the plots.

5.2 Theoretical and Empirical Reduction Factors

The theoretical factors obtained from the notebook are:

Method	Theoretical variance reduction
Antithetic variates	5.766
Control variate	11.420
Importance sampling	11.487

For $N = 10^6$, using the empirical variances from the 100 replications, the ratios relative to the standard method are approximately 8.00, 8.75, and 10.42, respectively. The discrepancy from the theoretical values is plausible because the empirical variance is itself estimated from only 100 realizations.

5.3 Execution-Time Comparison

Table 2: Execution times reported in the notebook, in seconds, for 100 replications.

N	Standard	Antithetic	Control variate	Importance sampling
100	0.011	0.005	0.011	0.012
1 000	0.109	0.044	0.079	0.113
10 000	1.613	0.459	0.583	1.151
100 000	8.374	6.288	5.652	13.125
1 000 000	64.550	51.944	66.741	130.495

Table 2 shows that a statistical comparison alone is not sufficient. At $N = 10^6$, importance sampling takes approximately 130.495 seconds, compared with 66.741 seconds for the control variate method and 51.944 seconds for the antithetic method. In the current code, this is also due to the additional cost of inverting the cumulative distribution function and the use of non-vectorized Python loops.

Therefore, when accuracy and computational cost are considered jointly, the control variate method appears particularly effective in this example: it achieves an RMSE very close to that of importance sampling in approximately half the observed execution time of the stored run.

The reported execution times may vary slightly across repeated runs, as they depend on the current computational load and execution environment

5.4 Final Results Expressed as Estimates of π

Table 3 converts the results at $N = 10^6$ from the integral I to the quantity $\pi = 6I$. The "mean estimate" column is the mean of the 100 estimates of π.

Table 3: Final comparison for $N = 10^6$, expressed directly in terms of π.

Method	Mean estimate	|bias|	RMSE	Variance	Var. red.	Time (s)
Standard	3.1415942	1.593×10^{-6}	1.229×10^{-4}	1.510×10^{-8}	1.00	64.55
Antithetic	3.1415940	1.380×10^{-6}	4.348×10^{-5}	1.888×10^{-9}	8.00	51.94
Control variate	3.1415920	6.665×10^{-7}	4.155×10^{-5}	1.726×10^{-9}	8.75	66.74
Importance sampling	3.1415937	1.026×10^{-6}	3.807×10^{-5}	1.449×10^{-9}	10.42	130.50

The mean values are all very close to π, as expected for unbiased estimators. The observed ranking in terms of final RMSE is:

$$\text{importance sampling} \lesssim \text{control variate} < \text{antithetic variates} < \text{standard Monte Carlo}.$$

However, the difference between importance sampling and the control variate method is much smaller than the difference in execution times.

5.5 Confidence-Interval Coverage

The empirical coverage percentages fluctuate around the nominal 95% value. Across the five values of N, the following ranges are observed:

> Standard: 93–98% (mean 94.8%); Antithetic: 92–100% (mean 96.6%); Control variate: 91–98% (mean 93.6%); Importance sampling: 94–97% (mean 95.2%).

Since each percentage is computed from only 100 replications, fluctuations of a few percentage points are normal. The result is therefore qualitatively consistent with the use of the normal approximation.

6 Discussion and Limitations

The experiment provides a simple illustration of three central ideas in variance reduction.

1. **Exploit negative dependence.** Antithetic variates construct two negatively correlated evaluations and reduce the dispersion of their average.
2. **Subtract a predictable component.** The control variate method exploits correlation with a quantity whose expected value is known.
3. **Sample where the integrand matters most.** Importance sampling changes the sampling density and corrects the weight of the observations.

The project nevertheless has several limitations, consistent with its educational nature.

- The integral has an elementary exact solution; Monte Carlo methods are not needed to compute it in practice. Precisely for this reason, however, the true value is available for measuring the error.
- Some "theoretical" quantities used in the code are computed numerically with `quad`. They serve as benchmarks and support the choice of parameters rather than being an essential part of the Monte Carlo estimator itself.
- The execution-time comparison depends on the specific implementation and hardware. A vectorized version could substantially change the relative timings.
- Empirical variances and coverage probabilities are estimated from 100 replications, so perfect agreement with theoretical values is not expected.
- Importance sampling is optimized only within a simple family of linear densities. No attempt is made to find an optimal density in a general sense.

These limitations are not shortcomings relative to the aim of the work: the purpose of the project is to illustrate concretely how the methods operate, not to construct a new algorithm or solve a competitive numerical problem.

7 Conclusions

The experiment confirms the basic theoretical behavior of the Monte Carlo method: the RMSE decreases approximately as $N^{-1/2}$. Variance-reduction techniques significantly improve the error constant without changing this convergence rate.

In the case considered, antithetic variates already provide a clear improvement and are simple to implement. The control variate method and importance sampling achieve theoretical variance reductions by a factor of approximately 11.4. Between these two, importance sampling produces the best empirical RMSE in the stored simulation, but the statistical advantage over the control variate method is marginal, while the execution time is approximately twice as large for $N = 10^6$.

For an undergraduate mathematics student, the main value of the project is therefore methodological: an elementary integration problem makes it possible to see probabilistic concepts in action that form the basis of much more general techniques used in advanced Monte Carlo methods.

References

[1] E. Gobet, *Monte-Carlo Methods and Stochastic Processes: From Linear to Non-Linear*, CRC Press, 2016. ISBN: 978-1-4987-4622-9. DOI: `https://doi.org/10.1201/9781315368757`.

[2] C. R. Harris et al., "Array programming with NumPy", *Nature*, 585, 357–362, 2020.

[3] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods*, 17, 261–272, 2020.

[4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment", *Computing in Science & Engineering*, 9(3), 90–95, 2007.